

SECURE WEB APPLICATION DEVELOPMENT & HARDENING

Flask Application with Full OWASP Top 10 Mitigations & SIEM Integration

Author: Nathan | University Student (IT/Cybersecurity) | Perth, Western Australia

Prepared for: Royal Australian Navy Information Warfare Officer Application Portfolio

Associated Activity: AAFC 721 Squadron (WA) Cadet Application & Skills Development

Date of Development & Testing: 7 July 2026

EXECUTIVE SUMMARY

This report documents the complete design, development, security hardening, and validation of a production-ready web application built with Flask. The application implements comprehensive security controls aligned with the OWASP Top 10 and the Australian Signals Directorate (ASD) Essential Eight mitigation strategies. It was developed as the capstone practical project in the cybersecurity home laboratory portfolio.

The application includes secure user authentication (Flask-Login + bcrypt), robust input validation and sanitisation (WTForms), security headers (Content-Security-Policy, HSTS, X-Frame-Options), centralised logging integrated with the Wazuh SIEM (Project #3), and file upload restrictions. The application was subjected to automated security scanning using OWASP ZAP. All identified vulnerabilities were remediated and re-tested until a clean scan result was achieved.

This project demonstrates end-to-end secure development capability — from initial architecture through coding, testing, hardening, and validation. It provides strong, tangible evidence of the secure coding and application security skills that are directly relevant to protecting Navy web-facing systems and APIs as an Information Warfare Officer.

1. INTRODUCTION AND OBJECTIVES

Many ADF systems expose web interfaces or APIs. An Information Warfare Officer must understand how to build, harden, and maintain secure applications so they do not become the weakest link in a defended network. This project addresses that requirement by building a realistic web application from the ground up with security as a first-class concern.

Objectives:

- Design and implement a functional Flask web application with user authentication and basic business logic.
- Implement all major OWASP Top 10 mitigations (injection, broken authentication, sensitive data exposure, XSS, insecure deserialisation, etc.).
- Integrate application logging with the central Wazuh SIEM for real-time security monitoring.
- Perform automated security testing with OWASP ZAP and remediate all findings.
- Map implemented controls to the ASD Essential Eight where applicable.
- Produce professional documentation suitable for an ADF Cyber Warfare Officer portfolio.

2. DETAILED IMPLEMENTATION AND PRACTICAL STEPS

2.1 Project Setup and Architecture

A new Flask project was initialised with a virtual environment:

```
python -m venv venv
source venv/bin/activate
pip install flask flask-login flask-wtf bcrypt sqlalchemy
```

The application followed a clean architecture with separate modules for authentication, routes, forms, and logging. SQLAlchemy was used as the ORM to eliminate raw SQL queries and prevent injection.

2.2 Secure Authentication Implementation

User registration and login were implemented using Flask-Login and bcrypt for password hashing:

```
from flask_login import LoginManager, login_user, logout_user, login_required
from werkzeug.security import generate_password_hash, check_password_hash
```

Password policy enforced: minimum 14 characters, complexity requirements, and bcrypt hashing with automatic salting. Session cookies were configured with secure=True, HttpOnly=True, and SameSite='Lax'. Flask-WTF provided automatic CSRF protection on all forms.

2.3 Input Validation and Sanitisation

All user input was validated using WTForms with strict validators:

```
from wtforms import StringField, PasswordField, validators
class LoginForm(FlaskForm):
    username = StringField('Username', [validators.Length(min=4, max=25)])
    password = PasswordField('Password', [validators.DataRequired()])
```

All form data was further sanitised on the server side. File uploads were restricted to specific extensions and size limits, with content-type validation and storage outside the web root.

2.4 Security Headers and Application Hardening

Security headers were added using Flask-Talisman or manual after_request handler:

```
response.headers['Content-Security-Policy'] = "default-src 'self'"
response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
response.headers['X-Frame-Options'] = 'DENY'
response.headers['X-Content-Type-Options'] = 'nosniff'
```

PowerShell execution was restricted where relevant, and the application was configured to run behind a reverse proxy (nginx) with additional hardening.

2.5 SIEM Logging Integration

Structured JSON logging was implemented and forwarded to the Wazuh agent:

```
import logging
from pythonjsonlogger import jsonlogger
logger = logging.getLogger(__name__)
handler = logging.FileHandler('/var/log/webapp/security.log')
handler.setFormatter(jsonlogger.JsonFormatter())
logger.addHandler(handler)
```

Failed logins, privilege escalation attempts, and file upload events were logged with user context and IP address. These events appeared in the Wazuh dashboard within seconds, enabling correlation with other lab activity.

2.6 OWASP Top 10 & Essential Eight Mapping

OWASP Top 10 / Essential Eight	Implementation in Flask App	Validation Method
A01:2021 – Broken Access Control	Flask-Login + @login_required decorator; role-based access on sensitive routes	ZAP forced browsing + manual session manipulation – blocked
A02:2021 – Cryptographic Failures	bcrypt + automatic salting; TLS enforced via reverse proxy	Traffic inspection – no plaintext credentials
A03:2021 – Injection	SQLAlchemy ORM (no raw SQL); WTForms validation + sanitisation	ZAP SQLi scanner – all payloads sanitised
A04:2021 – Insecure Design	Secure-by-default architecture; least privilege; input validation on all paths	Architecture review + threat modelling
A05:2021 – Security Misconfiguration	Security headers via after_request; debug=False in production; secret key from env var	ZAP + manual header inspection
A07:2021 – Identification & Auth Failures	Strong password policy; bcrypt; session management with secure flags	Credential stuffing simulation – rate limited + logged
Essential Eight – App Control	Only signed/approved Python packages in virtualenv; no arbitrary code execution paths	Attempted unsafe deserialisation / code exec – blocked
Essential Eight – User App Hardening	CSP headers; disabled unnecessary features; secure cookie flags	XSS payload testing – blocked by CSP and escaping

Table 1: OWASP Top 10 & Essential Eight Controls – Implementation and Validation Summary

2.7 Security Testing with OWASP ZAP

The running application was scanned using OWASP ZAP (both spider and active scan). Initial findings included reflected XSS in one search parameter and missing security headers. All issues were fixed within the same session:

- Reflected XSS → Fixed by proper template escaping and CSP
- Missing security headers → Added via after_request handler
- Insecure file upload → Added extension whitelist + size limit + content-type validation

A final clean ZAP scan was achieved and documented with before/after evidence.

3. LESSONS LEARNED AND ADF RELEVANCE

Building and hardening this application provided several operationally relevant insights:

- **Security must be built in, not bolted on:** Retrofitting security after the fact is far more expensive and less effective than designing it from the start.
- **Automated scanning finds what manual review misses:** ZAP discovered reflected XSS that had been overlooked during manual code review.
- **Logging to SIEM multiplies defensive value:** Security events from the application became immediately visible and correlatable with network and host activity in the central SIEM.
- **Secure development is a core Cyber Warfare Officer skill:** Many ADF systems have web components. Officers who can design, review, and harden applications provide direct value to mission assurance.
- **End-to-end portfolio value:** This project ties together the network segmentation (Project #1), offensive testing (Project #2), detection (Project #3), and investigation (Project #4) into one cohesive defensive capability demonstration.

4. CONCLUSION

A secure, functional Flask web application was successfully designed, built, hardened, and validated. All major OWASP Top 10 risks were addressed with verifiable controls, the application was integrated with the central SIEM, and a clean security scan was achieved. This project provides strong, practical evidence of secure development and application security capability directly aligned with the Navy Information Warfare Officer role. The artefacts (source code, configuration, ZAP reports, and this document) form a high-value component of the author's Navy application portfolio. Future work will include containerisation (Docker), CI/CD security scanning, and expansion to a microservices architecture with API security controls.

References: OWASP Top 10 (2021), Flask Security Documentation, Wazuh Documentation, ASD Essential Eight & ISM, Microsoft Secure Development Lifecycle guidance.